关键词唤醒

关键词唤醒
- 例程实验效果简介
- 代码解析

例程实验效果简介

这一节我们来介绍K230上的语音唤醒功能

本节例程代码位于：【 源码汇总 / 10.Media / 05.keyword_spotting.py 】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

点击CanMV IDE左下角的运行按钮

此时可以看到串行终端中持续输出Deactivated!

```
Deactivated!
total took 73.87 ms
Deactivated!
total took 298.68 ms
Deactivated!
total took 299.70 ms
Deactivated!
total took 299.86 ms
Deactivated!
total took 299.87 ms
Deactivated!
total took 299.83 ms
Deactivated!
total took 299.98 ms
Deactivated!
total took 299.88 ms
```

我们对着K230喊 【小楠小楠】，串行终端会输出 ====Detected XiaonanXiaonan!==== 说明检测到了唤醒词

```
total took 299.98 ms
Deactivated!
total took 299.88 ms
Deactivated!
total took 299.94 ms
====Detected XiaonanXiaonan!====
total took 1307.87 ms
Deactivated!
total took 22.50 ms
Deactivated!
```

> 该语音唤醒模型由 嘉楠科技 提供

代码解析

【完整的代码 & 双语注释请参考 [源码汇总 / 10.Media / 05.keyword_spotting.py]】

我们来分析这段语音唤醒（KWS）代码的结构：

1. 类设计架构

- 继承结构：`KWSApp` 继承自 `AIBase`
- 核心方法：

```
class KWSApp(AIBase):
    def __init__(self, kmodel_path, threshold, debug_mode=0)
    def preprocess(self, pcm_data)
    def postprocess(self, results)
```

2. 音频预处理流程

```
def preprocess(self, pcm_data):
    # PCM数据转换为float列表
    for i in range(0, len(pcm_data), 2):
        int_pcm_data = struct.unpack("<h", pcm_data[i:i+2])[0]  # 使用小端格式解析16位有符号整数
        float_pcm_data = float(int_pcm_data)                    # 转换为浮点数
        pcm_data_list.append(float_pcm_data)                    # 添加到列表中

    # 特征提取和张量转换
    mp_feats = aidemo.kws_preprocess(fp, pcm_data_list)[0]      # 调用aidemo模块的预处理函数提取音频特征
    mp_feats_np = np.array(mp_feats).reshape((1, 30, 40))       # 重塑数组形状为模型所需的输入维度
```

3. 后处理逻辑

```
def postprocess(self, results):
    logits_np = results[0]                                    # 获取第一个输出结果
（预测概率）
    self.cache_np = results[1]                                # 更新缓存数组，用于下
一次推理
    max_logits = np.max(logits_np, axis=1)[0]                 # 获取每个样本的最大概
率值
    # 阈值判断和结果输出
    if max_p > self.threshold and idx == 1:
        return 1
```

4. 音频流配置

```
input_stream = p.open(format=FORMAT,                          # 打开输入音频流，配置
格式、通道数、采样率等参数
                      channels=CHANNELS,
                      rate=SAMPLE_RATE,
                      input=True,
                      frames_per_buffer=CHUNK)
```

5. 主循环结构

```
while True:
    os.exitpoint()                                            # 检查是否有退出信号
    pcm_data = input_stream.read()                            # 读取音频数据
    res = kws.run(pcm_data)                                   # 运行关键词检测
    if res:
        # 播放回复音频
        wf = wave.open(reply_wav_file, "rb")                  # 打开回复音频文件
```